

Procedimiento Order():

```
//Creación de una LinkedList auxiliar
LinkedList linkedlist = nueva LinkedList();

//Mientras la cabeza de la lista actual no sea nula
mientras (this.head != nulo) hacer:

    //Asignación de la cabeza de la lista actual a un puntero
    LinkedListNode pointer = this.head;

    //Inicialización de los nodos mínimo y previo al mínimo como nulos
    LinkedListNode prevMin = nulo;
    LinkedListNode min = pointer;

    //Bucle para encontrar el mínimo de la lista
    mientras (pointer.next != nulo) hacer:

        //Obtención de los valores enteros de los nodos a comparar
        entero ja = (entero) pointer.next.data;
        entero ja2 = (entero) min.data;

        //Comparación entre los valores enteros de los nodos
        si (ja < ja2) entonces:

            //Asignación del puntero al nodo anterior al mínimo encontrado
            prevMin = pointer;

            //Asignación del puntero al nodo mínimo encontrado
            min = pointer.next;

        //Asignación del puntero al siguiente nodo de la lista
        pointer = pointer.next;

    //Eliminación del nodo mínimo encontrado de la lista actual
    si (prevMin != nulo) entonces:
        prevMin.setNext(min.next);
    sino:
        this.setHead(min.next);

    //Creación de un nuevo nodo con el valor mínimo encontrado
    LinkedListNode aux = nuevo LinkedListNode(min.data);
```

```
//Añadir el nuevo nodo a la LinkedList auxiliar  
linkedlist.addLast(aux.data);  
linkedlist.addFirst(aux);
```

```
//Imprimir el valor del nuevo nodo  
imprimir("{ " + aux.data + " } ");
```

fin mientras

```
//Imprimir un salto de línea para separar el resultado de la ejecución del método  
imprimir("");  
imprimir(""); //Dos veces para asegurar dos saltos de línea consecutivos
```